

Take-Home Assignment: Patient Portal Web App

Time estimate: 4 hours. Solve as many of the problems below as you can in that window.

AI assistance is encouraged. Use whatever tools help you move fast. If you use agentic coding, include a transcript of your session with your submission.

What to build

A patient portal lets patients view and book appointments online instead of calling the front desk, while providers manage those requests from the other side. Build a minimal version: two user types, a **patient** and a **provider**, selectable with a simple role switcher. No real auth needed.

This is the baseline. Once it works, the problems in the next section are where the real assignment is.

Patient view

1. **Appointment list.** Display upcoming appointments with date, time, provider name, appointment type, and current status. Seed a few mock appointments.
2. **Request an appointment.** A form to submit a new appointment request (date/time preference, reason for visit). New requests start in `pending` status. No real scheduling logic needed. Just persist the request and show it in the list.
3. **Cancel an appointment.** A patient can cancel a confirmed appointment. Pending appointments cannot be cancelled.

Provider view

1. **Appointment list.** The same list from the provider's side. Shows all appointments assigned to them.

2. **Confirm or reschedule.** The provider can confirm a pending appointment, or reschedule it by updating the date/time. Confirmed appointments move to `confirmed` status.

Appointment status flow: `pending` → `confirmed` → `cancelled`

Problems to solve

With the baseline working, work through the problems below. Each is a situation, not a feature spec, and none has a single right answer. We care about how you frame it, what you decide, and why. Solve as many as you can in the 4-hour window, in any order. If you leave any unsolved, we can talk through it in the review.

Problem 1

A provider opens an appointment and moves it from 2:00 to 4:00. At that same second the patient is looking at their screen, which still says 2:00, and clicks Cancel or Confirm. The patient did nothing wrong; they acted on what was correct a moment earlier. So what should happen? Decide, and make the app do that. Think through what the patient sees, what the provider sees, and what actually gets saved.

In the review: why that behavior over the alternatives, and where in the stack you enforced it.

Problem 2

Weeks later, someone asks why an appointment is on Wednesday when the patient swears they booked Monday. When did it change? Who changed it? What was it before? Build the app so those questions can be answered. Rescheduling, cancelling, or confirming shouldn't quietly erase whatever came before.

In the review: how this fits with the rest of your model (times, status, edits), and whether it's one mechanism or several bolted together.

Problem 3

When a provider confirms an appointment, the patient should get notified by email or text. You don't need to send anything real; a stub that logs "would send X to Y"

is fine. The catch is that confirming can't get slow, and can't fail, just because the notification step is slow or broken. The confirm needs to succeed on its own.

In the review: what you built, why it's good enough for this exercise, and what you'd change for real traffic.

Problem 4

A provider can't be in two places at once, so two overlapping confirmed appointments for the same provider should never exist. That includes the ugly case: two pending requests confirmed at almost the same instant. Make it impossible, not just unlikely. Hiding the second button in a busy UI doesn't count.

In the review: where exactly you enforced this and why that layer, and what happens under two simultaneous confirms.

Technical requirements

- **Any stack.** Use whatever language and framework fit the job best. We'll ask you to justify the choice.
 - **Persistent enough.** Data should survive a page refresh. An in-memory store or SQLite is fine.
 - **Runnable locally.** Include a short README with setup instructions. It should start with one or two commands.
 - **Keep it simple.** We're not testing for scale, so skip the microservices, the queues, and the auth systems. The simplest thing that solves the problem is the right answer, and we're grading on that. If a problem pulls you toward something heavy, the version we want is usually the small one, plus a sentence on what you'd do differently once it actually has to scale.
-

Submission

Submit your solution **by email**, as a link to a public git repo or a zip file. Include a README with setup instructions and any notes you want to share. If you used agentic coding, include a transcript of your session.

During the review

Plan to present your app to us. Give a short demo of it running, then walk us through the decisions you made and why you made them. After that, take us through the code and how it's structured. Expect us to dig into the problems you solved, how you'd extend the app, and how it would hold up as requirements and scale grow.